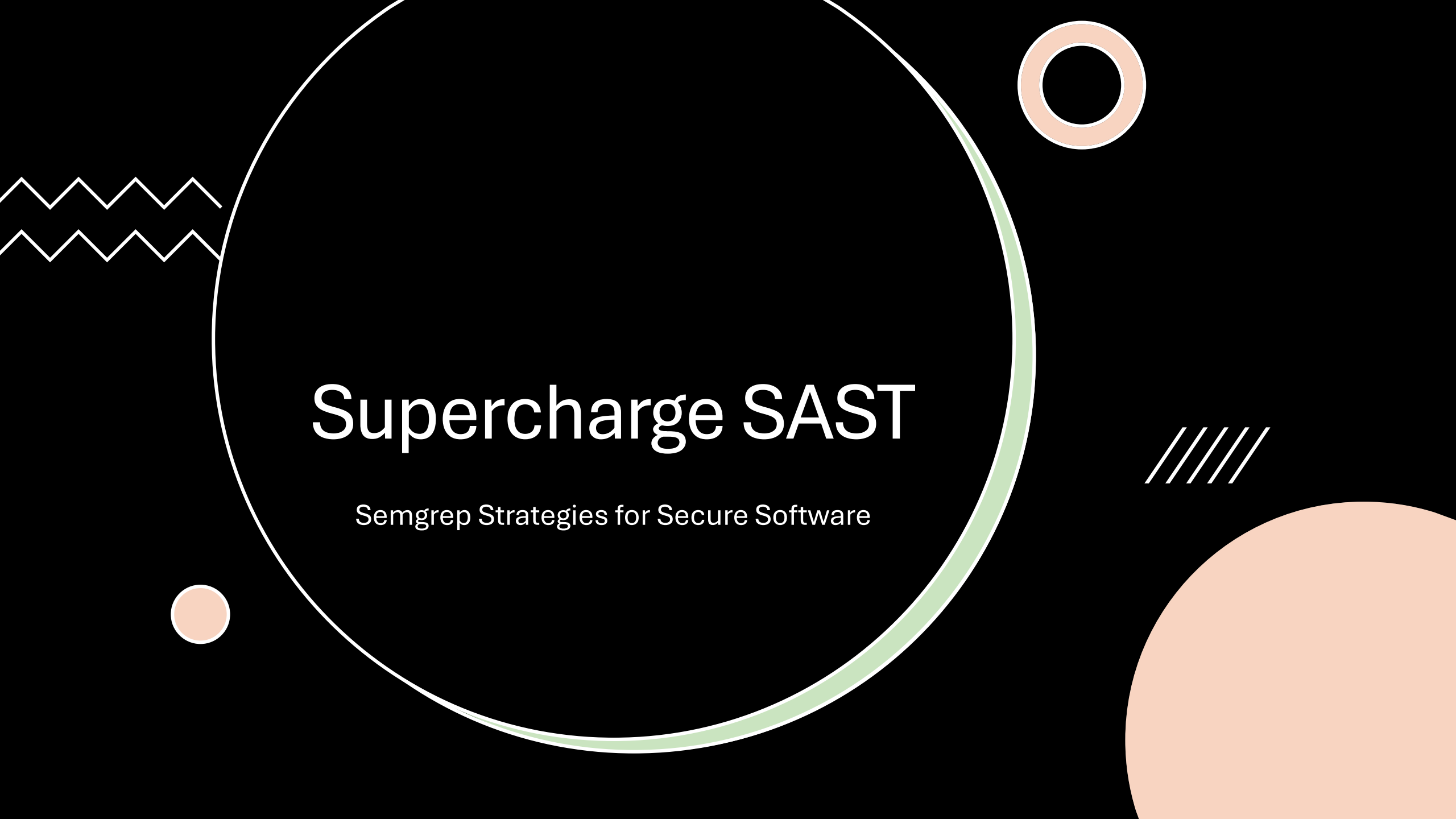
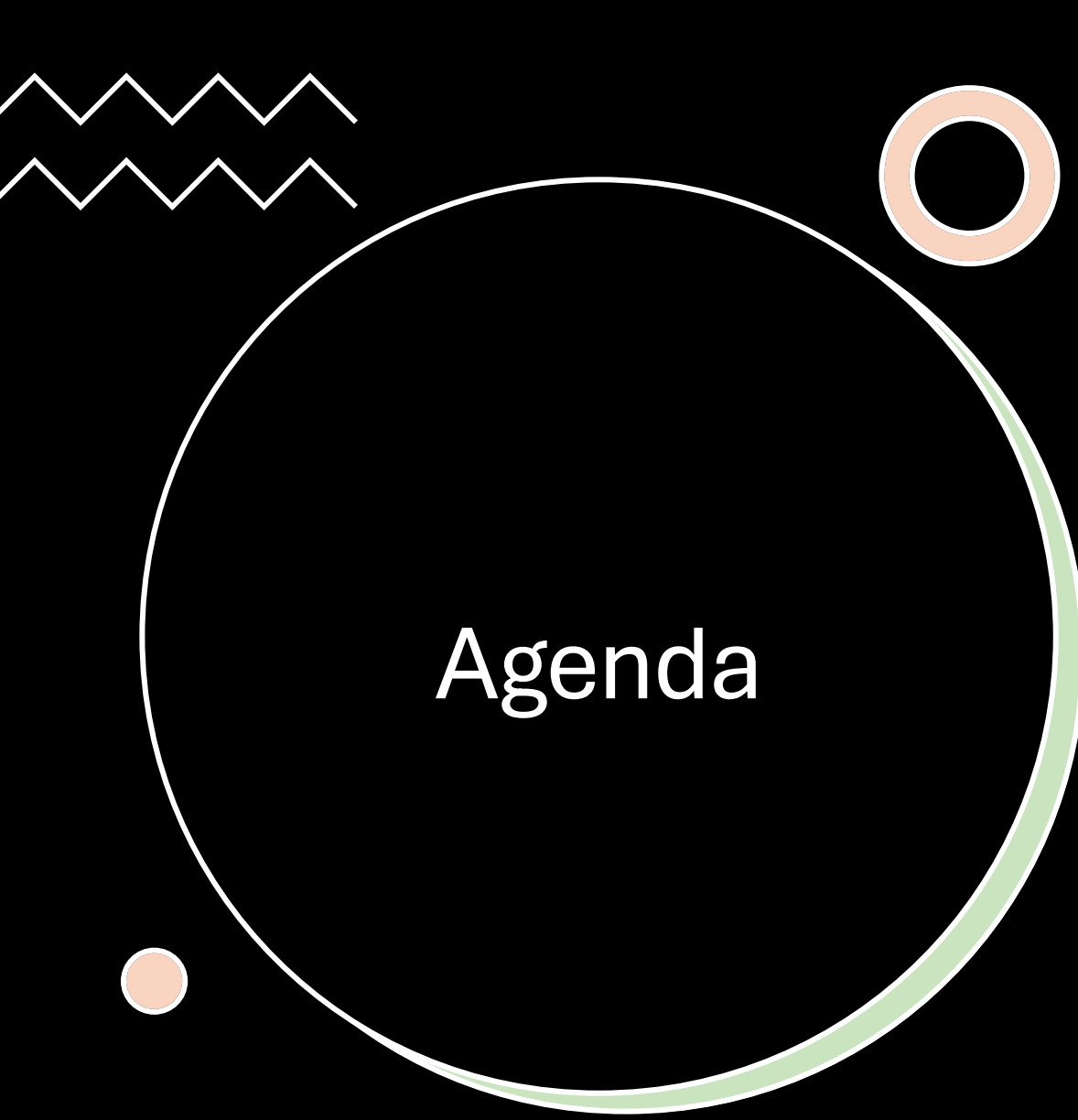




Supercharge SAST

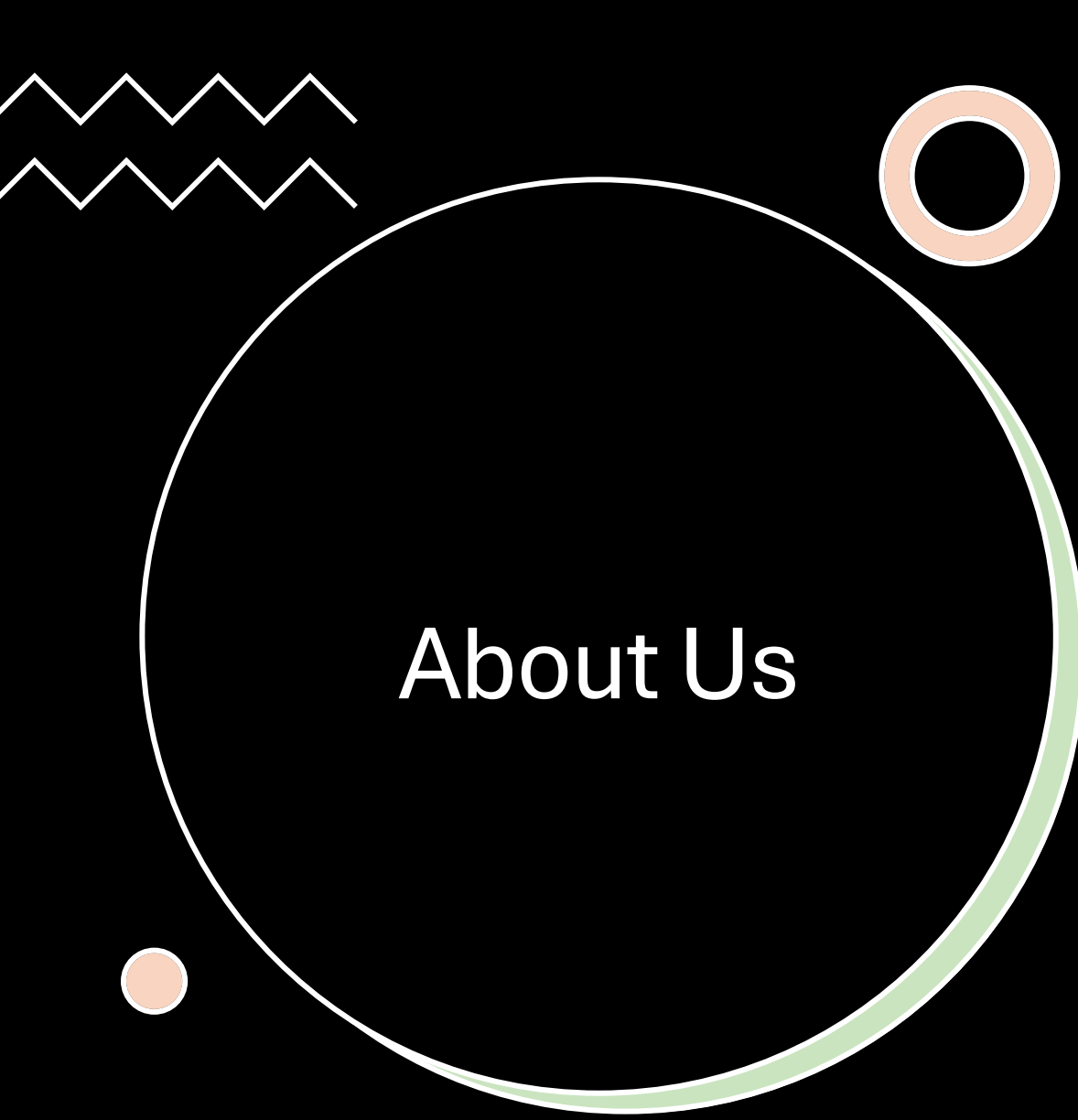
Semgrep Strategies for Secure Software



Agenda

- SAST and AppSec
- Getting started with Semgrep
- Writing custom rules
- Advanced Semgrep features
- Practical exercises
- Wrap-up
- Q&A





About Us

Software Security Engineering
@ Microsoft

Arjun

<https://www.linkedin.com/in/247arjun>

Yun

<https://www.linkedin.com/in/yun-ke-a126723/>

Parsia

<https://www.linkedin.com/in/parsiya/>





SAST and Application Security

MODULE 1



SAST and Application Security



Understanding SAST:
Concepts and significance
in securing software.



The role of SAST in the
Software Development Life
Cycle (SDLC).



What is SAST?

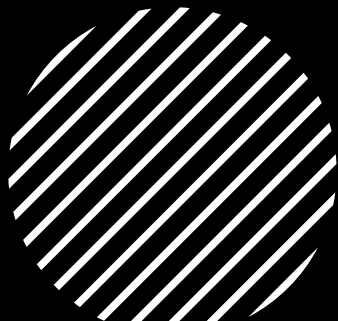
Static Application Security Testing (SAST) is a testing methodology that analyzes source code to find security vulnerabilities in applications

- Depends on patterns to define a vulnerability, also known as **signatures**

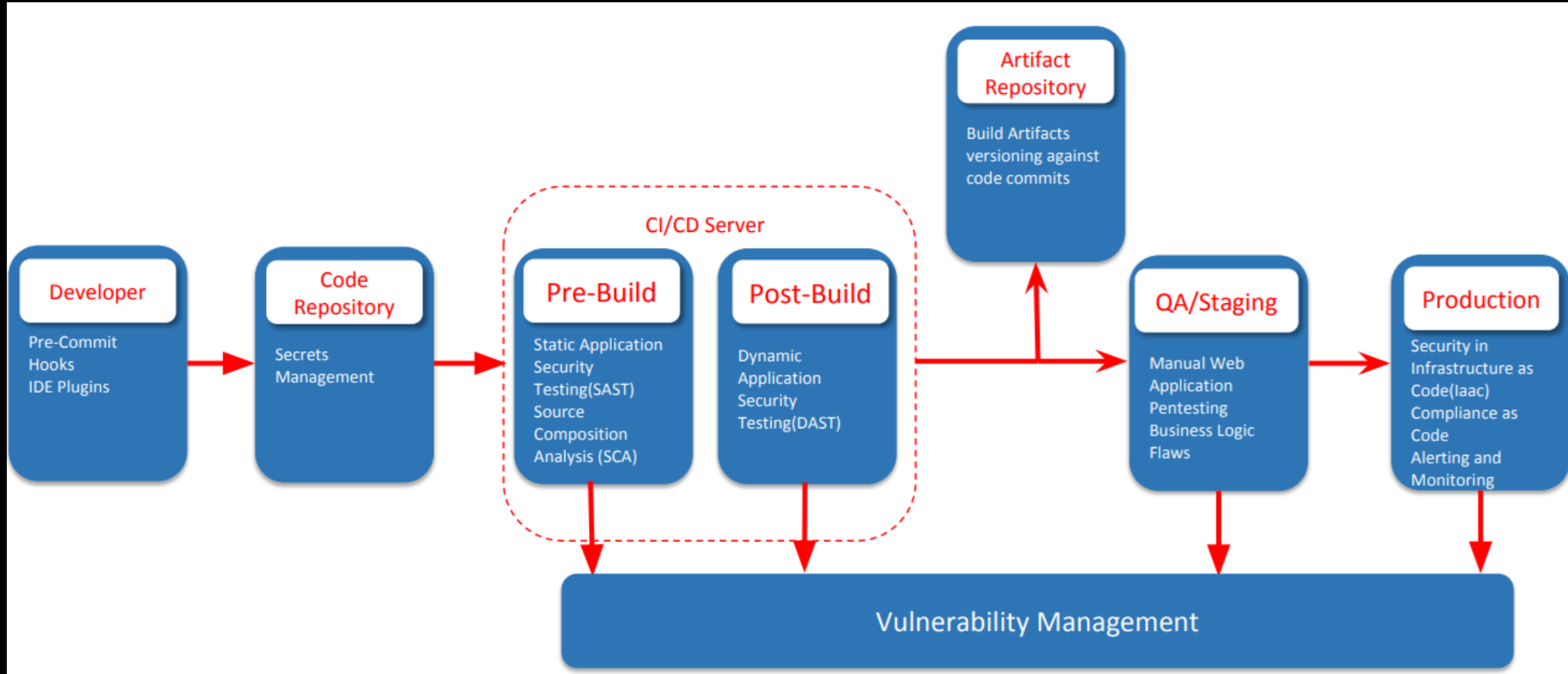
SAST can be used in real-time, as the developers code, it can be integrated into the DevOps cycle, or it can be executed offline.

As it is heavily dependent on signatures, it could lead to false positives if the signature base are not verified and tested to reduce de false alerts rate.

- LLMs can also be leveraged to reduce the false positive/negative rates



Typical SDLC





Getting Started with Semgrep

MODULE 2



Getting Started with Semgrep



Installing and configuring
Semgrep.



Navigating the Semgrep
ecosystem: CLI and
Playground.



Installing and getting ready

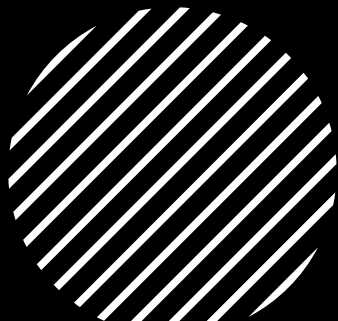
Install Semgrep on Linux, Windows and macOS

Pre-Requisite: Having Python 3.8 pre-installed

With all those OS, we can install Semgrep with PIP (make sure you have PIP installed)

`python3 -m pip install semgrep`

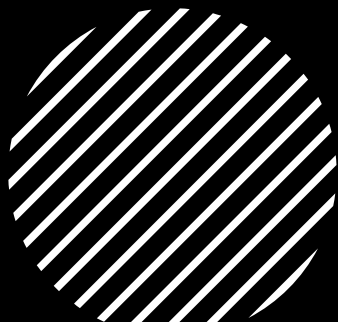
Installing Semgrep using other methods (Homebrew on macOS) or use the Semgrep Docker container to run it are options, but those methods won't be covered on this workshop.





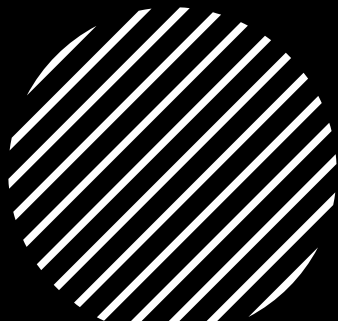
Semgrep... Installing and getting ready

- Two use modes:
 - Online rules with Semgrep web interface (semgrep.dev)
 - The GUI helps with analysis over the findings
 - It has a free tier, but not for the PRO ruleset
 - Local rules
 - You can use the default ruleset, and add local rules set you built or downloaded
 - It generates text-formatted outputs with the findings
 - It does not provide a GUI to help with analysis... (but you can always feed the results into an ELK/Splunk for analysis...)
- We will focus on features available on the Open Source ONLY!



Navigating the Semgrep ecosystem: CLI and Playground

- Basic CLI Command:
 - `semgrep scan [options] [targets]`
- Some interesting Semgrep CLI options:
 - `--config auto` – Downloads the relevant rules for the current project from Semgrep opensource rules repository
 - `--config <path_to_rules>` - Points to local custom rulesets
 - Output formats
 - `--json` – Output to JSON formatted file
 - `--text` – Output to a text formatted file
 - `--vim` – Output to a vim-friendly formatted file
 - `--sarif` – Output to a sarif formatted file



Semgrep Playground

(<https://semgrep.dev/playground/new>)

The screenshot displays the Semgrep Playground web interface. The top navigation bar includes links for Registry, Playground, Products, Pricing, and Docs, along with a 'Sign in / Sign up free' button. The main interface is divided into three primary sections:

- Left Panel (Library):** Features a search bar with the placeholder 'e.g.: python.flask', a large blue arrow icon, and a 'Sign in required' message stating 'Sign in to view and search the registry and your saved rules.' with a 'Sign in' button.
- Middle Panel (Rule Editor):** Titled 'untitled_rule', it contains a 'search' tab with a 'pattern' input field containing the code `print("...")`. Below this is a 'Rule info' section with fields for 'Rule ID' (untitled_rule), 'Language' (Python), 'Severity' (ERROR), and a 'Message' field showing 'Semgrep found a match'.
- Right Panel (Test Code):** Contains a 'test code' tab with a code editor showing a Python script. The script includes a welcome message, a loop of print statements, and a commented-out line. A 'Run' button is located at the bottom right of this panel.

At the bottom of the interface, a status bar indicates 'Run your rule to see matches.'



Writing Custom Semgrep Rules

MODULE 3



Writing Custom Semgrep Rules

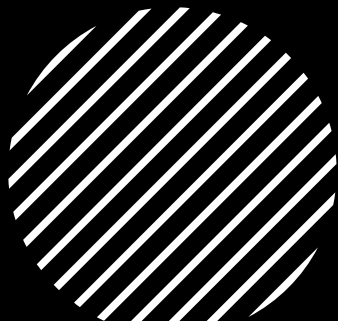


Deep dive into Semgrep's rule syntax and pattern matching.



Semgrep Rules

- *A rule is a specification of the patterns* that Semgrep must match to the code to generate a finding.
- Semgrep Rules are written in YAML
- Semgrep supports multiple languages
 - e.g.: Python, Java, C#, JavaScript, Go, etc..
- Multiple modes
 - Search (default)
 - Taint
 - Join
 - Extract





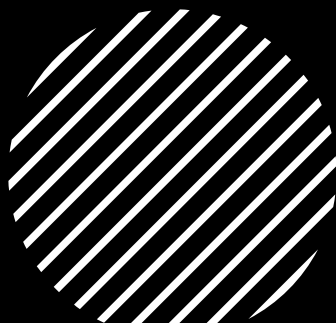
Schema – Required Fields

Field	Type	Description
<code>id</code>	<code>string</code>	Unique, descriptive identifier, for example: <code>no-unused-variable</code>
<code>message</code>	<code>string</code>	Message that includes why Semgrep matched this pattern and how to remediate it. See also Rule messages .
<code>severity</code>	<code>string</code>	One of the following values: <code>INFO</code> (Low severity), <code>WARNING</code> (Medium severity), or <code>ERROR</code> (High severity). The <code>severity</code> key specifies how critical are the issues that a rule potentially detects. Note: Semgrep Supply Chain differs, as its rules use CVE assignments for severity. For more information, see Filters section in Semgrep Supply Chain documentation.
<code>languages</code>	<code>array</code>	See language extensions and tags
<code>pattern</code> *	<code>string</code>	Find code matching this expression
<code>patterns</code> *	<code>array</code>	Logical AND of multiple patterns
<code>pattern-either</code> *	<code>array</code>	Logical OR of multiple patterns
<code>pattern-regex</code> *	<code>string</code>	Find code matching this PCRE2 -compatible pattern in multiline mode



Schema – Optional Fields

Field	Type	Description
<code>options</code>	object	Options object to enable/disable certain matching features
<code>fix</code>	object	Simple search-and-replace autofix functionality
<code>metadata</code>	object	Arbitrary user-provided data; attach data to rules without affecting Semgrep behavior
<code>min-version</code>	string	Minimum Semgrep version compatible with this rule
<code>max-version</code>	string	Maximum Semgrep version compatible with this rule
<code>paths</code>	object	Paths to include or exclude when running this rule



The below optional fields must reside underneath a `patterns` or `pattern-either` field.

Field	Type	Description
<code>pattern-inside</code>	string	Keep findings that lie inside this pattern

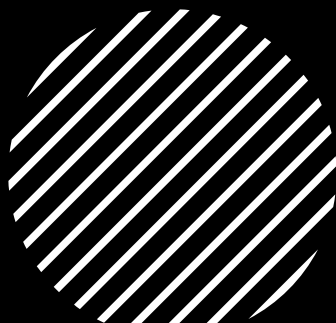
The below optional fields must reside underneath a `patterns` field.

Field	Type	Description
<code>metavariable-regex</code>	map	Search metavariables for Python re compatible expressions; regex matching is unanchored
<code>metavariable-pattern</code>	map	Matches metavariables with a pattern formula
<code>metavariable-comparison</code>	map	Compare metavariables against basic Python expressions
<code>pattern-not</code>	string	Logical NOT - remove findings matching this expression
<code>pattern-not-inside</code>	string	Keep findings that do not lie inside this pattern
<code>pattern-not-regex</code>	string	Filter results using a PCRE2 -compatible pattern in multiline mode



Semgrep Rule Syntax

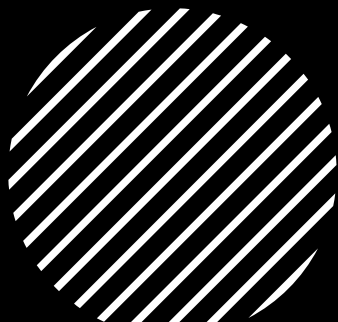
```
1 rules:
2   - id: detect-console-writeline
3     patterns:
4       - pattern: Console.WriteLine($X);
5       message: "Avoid using Console.WriteLine in production code."
6       severity: WARNING
7       languages: Required fields
8         - csharp
9       metadata: Optional fields
10        category: best-practice
11        technology: dotnet
12
```





Pattern Syntax

- Ellipsis Operator (...)
- Metavariables (\$VARNAME)





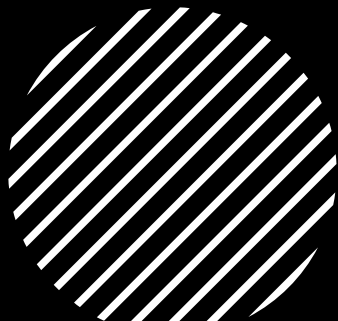
Ellipsis Operator ...

- Abstracts away a sequence of zero or more items such as arguments, statements, parameters, fields, characters.

```
1 rules:
2   - id: find-foo-calls
3     patterns:
4       - pattern: foo(...)
5     message: "Found a call to foo"
6     languages:
7       - python
8     severity: WARNING
9
```



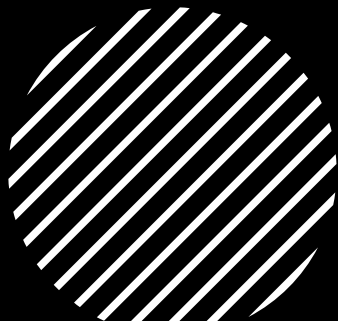
```
1 def foo(*args):
2     print(f"Arguments received: {args}")
3
4 # Call foo with different number of arguments, including
5 # strings and other objects
6 foo(1)
7 foo(1, 2)
8 foo(1, "two", 3.0)
9 foo(1, "two", 3.0, [4, 5])
10 foo(1, "two", 3.0, [4, 5], {"six": 6})
```

 Matches



Ellipsis Operator ...

- Exercise - (aka.ms/semgrep-exercises)
 - <https://github.com/django/django/blob/main/django/http/request.py>
 - Flag and return the entire function body for
 - *def get_host(self):*
 - *def _get_full_path():* – not knowing the function's arguments





Ellipsis Operator ...

- Answers

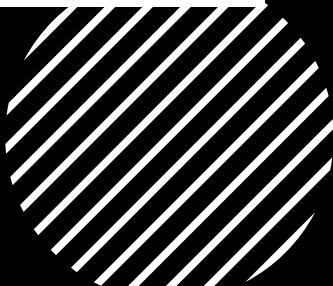
- Flag and return the entire function body for

- `def get_host(self):`

```
pattern: |-  
|   def get_host(self):  
|   |   ...
```

- `def _get_full_path` – *not knowing the function's arguments*

```
pattern: |-  
|   def _get_full_path(...):  
|   |   ...
```



Metavariables - \$VARIABLE_NAME

- Metavariables are an abstraction to match code when you don't know the value or contents ahead of time

```
1 rules:
2   - id: find-all-functions
3     patterns:
4       - pattern: |
5         |   def $FUNC(...):
6         |       ...
7         |   message: "Found a function definition"
8         |   severity: INFO
9         |   languages: [python]
10
```

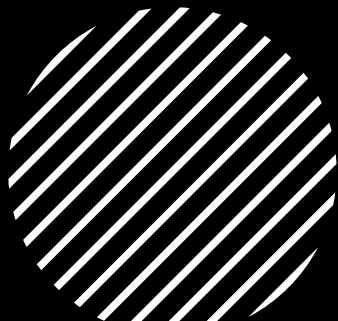


```
1 # Define a function to add two numbers
2 def add(a, b):
3     return a + b
4
5 # Define a function to subtract two numbers
6 def subtract(a, b):
7     return a - b
8
9 # Define a function to multiply two numbers
10 def multiply(a, b):
11     return a * b
12
13 # Define a function to divide two numbers
14 def divide(a, b):
15     if b != 0:
16         return a / b
17     else:
18         return "Division by zero is not allowed"
19
20 # Invoke the functions and print the results
21 print("Addition of 5 and 3:", add(5, 3))
22 print("Subtraction of 5 and 3:", subtract(5, 3))
23 print("Multiplication of 5 and 3:", multiply(5, 3))
24 print("Division of 5 by 3:", divide(5, 3))
```




Metavariables - \$VARIABLE_NAME

- Exercise - (aka.ms/semgrep-exercises)
 - <https://github.com/django/django/blob/main/django/http/request.py>
 - Flag and return the entire function bodies that contain the line
 - *return self.scheme == "https"*
 - *parser = MultiPartParser(...)*





Metavariables - \$VARIABLE_NAME

- Answers

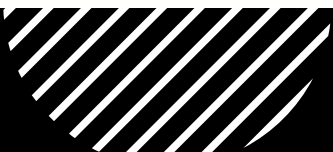
- Flag and return the entire function bodies that contain the line
 - *return self.scheme == "https"*

```
pattern: |-  
  def $FUNC(...):  
    return self.scheme == "https"
```

```
pattern: |-  
  def $FUNC(...):  
    ...  
    return self.scheme == "https"  
    ...
```

- *parser = MultiPartParser(META, post_data, self.upload_handlers, self.encoding)*

```
pattern: |-  
  def $FUNC(...):  
    ...  
    parser = MultiPartParser(META, post_data, self.upload_handlers, self.encoding)  
    ...
```





Pattern Matching Operators

pattern

patterns

pattern-either

pattern-regex

pattern-not-regex

focus-metavariable

metavariable-regex

metavariable-pattern

metavariable-comparison

pattern-not

pattern-inside

pattern-not-inside

Metavariable matching

Metavariables in logical ANDs

Metavariables in logical ORs

Metavariables in complex logic

options

fix

metadata

min-version and max-version

category

paths

Excluding a rule in paths

Limiting a rule to paths

Other examples

Complete useless comparison



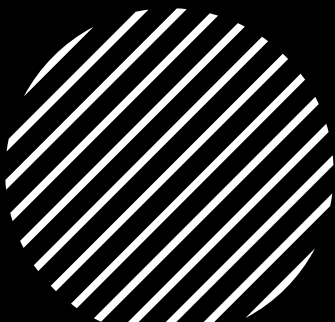
Pattern

- The *pattern* operator looks for code matching its expression.

```
1 rules:
2 - id: find-logger-log-calls
3   patterns:
4     - pattern: logger.log(...)
5     message: "Found a call to logger.log"
6     languages: [python, javascript, java, go]
7     severity: WARNING
8
```



```
1 import logging
2
3 # Create a logger
4 logger = logging.getLogger(__name__)
5
6 def example_function():
7     logger.log(logging.INFO, "This is a log message") # Matches the pattern
8     logger.info("This is an info message") # Does not match the pattern
9
10 def another_function():
11     logger.log(logging.ERROR, "This is an error message") # Matches the pattern
12     print("This is a print statement") # Does not match the pattern
13
14 if __name__ == "__main__":
15     example_function()
16     another_function()
17
```



Patterns

- Performs a logical AND operation on one or more child patterns.
- Chaining multiple patterns together

```
1 rules:
2   - id: avoid-logging-sensitive-info
3     patterns:
4       - pattern-either:
5         - pattern: |
6           $LOGGING_FUNCTION($PASSWORD_VAR)
7         - pattern: |
8           $LOGGING_FUNCTION($ANY_VAR, $PASSWORD_VAR)
9       - pattern-inside: |
10        def $FUNC(...):
11          ...
12          $LOGGING_FUNCTION(...)
13        ...
14      message: "Avoid logging sensitive data such as usernames and passwords."
15      severity: ERROR
16      languages:
17        - python
18      metadata:
19        category: security
20        technology: logging
```

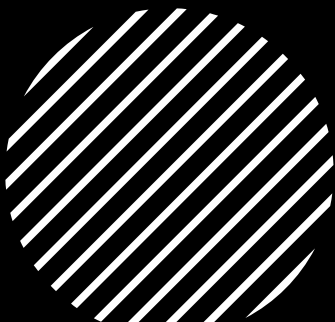


```
1
2 import logging
3
4 def process_user_data(username, password):
5     logging.info(f"Processing data for user: {username}")
6     logging.info(f"User password: {password}")
7
8 def another_function():
9     password = "super_secret_password"
10    logging.debug("Debugging password issue: %s", password)
11
12 if __name__ == "__main__":
13     process_user_data("user123", "my_password")
14     another_function()
15
```



Patterns

- Exercise - (aka.ms/semgrep-exercises)
 - <https://github.com/django/django/blob/main/django/http/request.py>
 - Using 1 rule, flag and return the line of code
 - That is inside a function named *read*
 - And sets the *_read_started* attribute to *True*

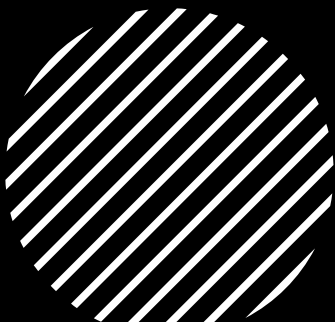




Patterns

- Answer
 - Using 1 rule, flag and return the line of code
 - That is inside a function named *read*
 - And sets the *_read_started* attribute to *True*

```
patterns:  
- pattern: |  
  |   def read(...):  
  |   ...  
- pattern: $SELF._read_started = True
```





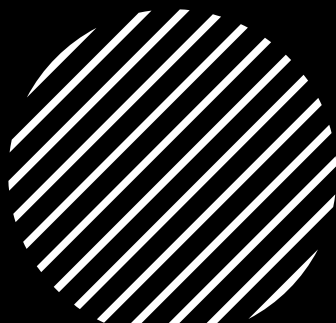
Pattern-Either

- Performs a logical OR operation on one or more child patterns.
- Chaining multiple patterns together

```
1 rules:
2   - id: insecure-code-detection
3     patterns:
4       - pattern-either:
5         - pattern: |
6           def $FUNC(...):
7             ...
8         - pattern: |
9           class $CLASS(...):
10            def $METHOD(...):
11              ...
12 message: "Potential insecure function or method definition"
13 languages: [python]
14 severity: ERROR
15
```



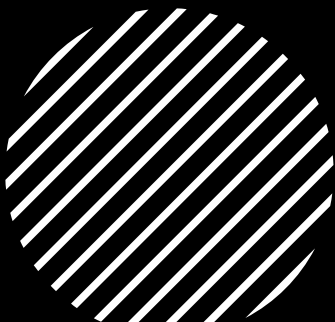
```
1 # Insecure function definition
2 def insecure_function(password):
3     print(f"Password: {password}")
4
5 # Insecure class definition
6 class InsecureClass:
7     def __init__(self, secret_key):
8         self.secret_key = secret_key
9
10    def display_secret_key(self):
11        print(f"Secret Key: {self.secret_key}")
12
13 print('Semgrep wont match this line')
```





Pattern-Either

- Exercise - (aka.ms/semgrep-exercises)
 - <https://github.com/django/django/blob/main/django/http/request.py>
 - Using 1 rule, flag and return both functions
 - A function named *read*
 - A function named *readline*

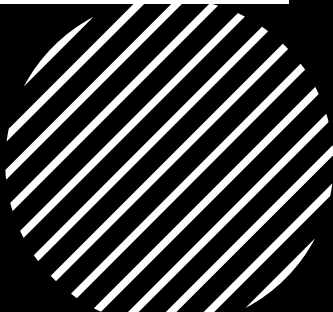




Pattern-Either

- Answer
 - Using 1 rule, flag and return both functions
 - A function named *read*
 - A function named *readline*

```
pattern-either:  
  - pattern: |-  
    |   def read(...):  
    |   ...  
  - pattern: |-  
    |   def readline(...):  
    |   ...
```



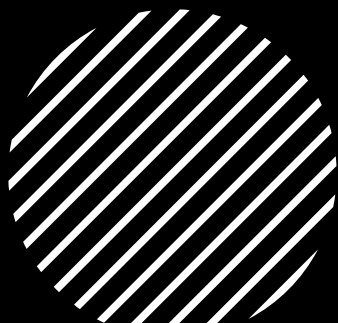


Pattern-Not

- The *pattern-not* operator is the opposite of the pattern operator.
- Finds code that does **NOT** match its expression

```
1 rules:
2   - id: ssrf-detection-python
3     patterns:
4       - pattern: |
5         requests.get($URL)
6       - pattern-not: |
7         requests.get('http://trusted-domain.com')
8     message: "Potential SSRF vulnerability detected"
9     severity: ERROR
10    languages: [python]
11    metadata:
12      cwe: "CWE-918"
13      owasp: "A10:2017"
14
```

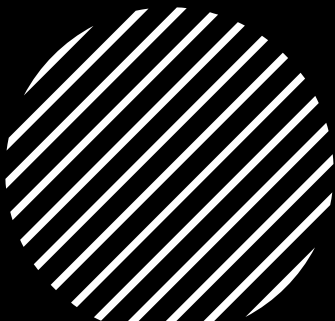
```
1 import requests
2
3 # Example of potential SSRF vulnerability
4 url = "http://example.com"
5 response = requests.get(url)
6
7 # This line would be excluded by the pattern-not rule
8 trusted_response = requests.get('http://trusted-domain.com')
9
```





Pattern-Not

- Exercise - (aka.ms/semgrep-exercises)
 - <https://github.com/django/django/blob/main/django/http/request.py>
 - Using 1 rule, flag and return both functions
 - All functions named *encoding*
 - But not those that are one-liners
 - *return self._encoding*
 - *self._encoding = value*





Pattern-Not

- Answer
 - Using 1 rule, flag and return both functions
 - All functions named *encoding*
 - But not those that are one-liners
 - *return self._encoding*
 - *self._encoding = value*

```
patterns:  
- pattern: |-  
  |   |   |  
  |   |   | def encoding(...):  
  |   |   |   |  
  |   |   |   | ...  
- pattern-not: |-  
  |   |   |  
  |   |   | def encoding(...):  
  |   |   |   |  
  |   |   |   | return self._encoding  
- pattern-not: |-  
  |   |   |  
  |   |   | def encoding(...):  
  |   |   |   |  
  |   |   |   | self._encoding = value
```

Pattern-Inside

- Matches findings that reside within its expression

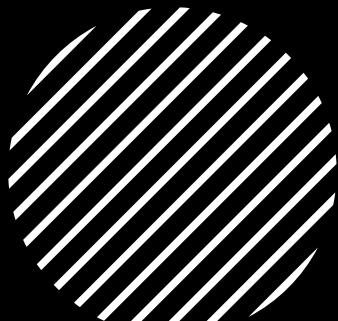
```
1 rules:
2   - id: avoid-logging-sensitive-info
3     patterns:
4       - pattern-either:
5         - pattern: |
6             $LOGGING_FUNCTION($PASSWORD_VAR)
7         - pattern: |
8             $LOGGING_FUNCTION($ANY_VAR, $PASSWORD_VAR)
9       - pattern-inside: | ←
10         def $FUNC(...):
11             ...
12             $LOGGING_FUNCTION(...)
13         ...
14     message: "Avoid logging sensitive data such as usernames and passwords."
15     severity: ERROR
16     languages:
17       - python
18     metadata:
19       category: security
20       technology: logging
21
```

```
1
2 import logging
3
4 def process_user_data(username, password):
5     logging.info(f"Processing data for user: {username}")
6     logging.info(f"User password: {password}")
7
8 def another_function():
9     password = "super_secret_password"
10    logging.debug("Debugging password issue: %s", password)
11
12 if __name__ == "__main__":
13     process_user_data("user123", "my_password")
14     another_function()
15
```



Pattern-Inside

- Exercise - (aka.ms/semgrep-exercises)
 - <https://github.com/django/django/blob/main/django/http/request.py>
 - Using 1 rule, flag and return
 - All instances of *if hasattr(...): ...*
 - That are inside a function named *encoding*

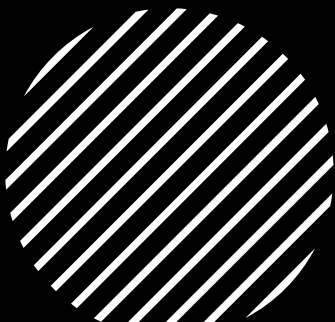




Pattern-Inside

- Answer
 - Using 1 rule, flag and return
 - All instances of *if hasattr(...): ...*
 - That are inside a function named *encoding*

```
patterns:  
- pattern: "if hasattr(...): ..."  
- pattern-inside: |-  
  def encoding(...):  
    ...
```





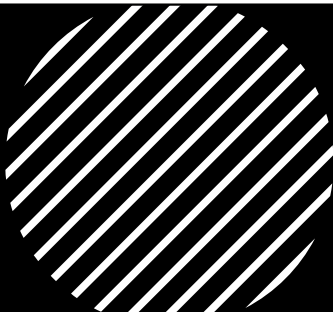
Metavariable regex

- Searches metavariables for a PCRE2 regular expression
- Useful in filtering results based on a metavariable's value

```
1 rules:
2   - id: example-rule
3     patterns:
4       - pattern: |
5         |   def $FUNC(...):
6         |     ...
7       - metavariable-regex:
8         |   metavariable: $FUNC
9         |   regex: 'test_.*'
10    message: "Function name should not start with 'test_'"
11    languages: [python]
12    severity: WARNING
13
```



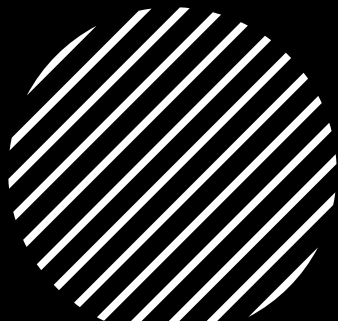
```
1 def test_example_function():
2   print("This function name starts with 'test_' and should trigger the
3     Semgrep rule.")
4
5 def example_function():
6   print("This function name does not start with 'test_' and should not
7     trigger the Semgrep rule.")
```





Metavariable-regex

- Exercise - (aka.ms/semgrep-exercises)
 - <https://github.com/django/django/blob/main/django/http/request.py>
 - Using 1 rule, flag and return
 - All function names that begin with *get_full_path*

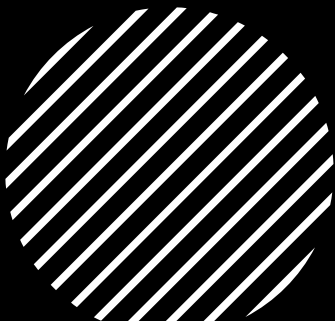


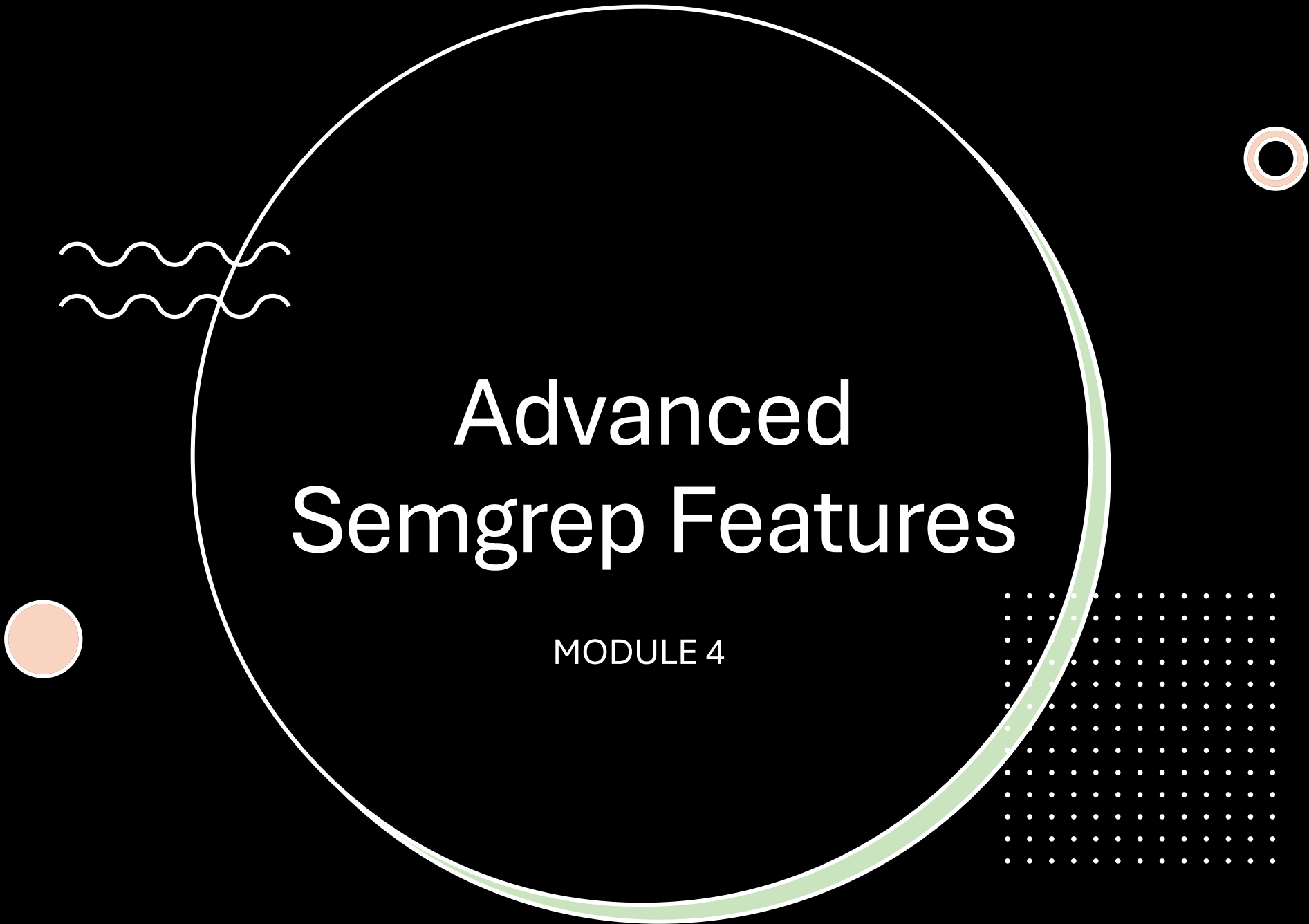


Metavariable-regex

- Answer
 - Using 1 rule, flag and return all function names that begin with *get_full_path*

```
patterns:  
  - pattern: |  
    |   def $FUNC(...):  
    |   |   ...  
  - metavariable-regex:  
    |   metavariable: $FUNC  
    |   regex: 'get_full_path.*'
```





Advanced Semgrep Features

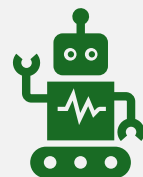
MODULE 4



Advanced Semgrep Features



Overview of advanced Semgrep features: Taint mode, and editor integration.



Leveraging Semgrep findings for LLM-based code analysis.

Taint Mode

- Tracks the flow of untrusted, or **tainted** data throughout the body of a function or method

```
1 rules:
2 - id: tainted-code-exec
3   mode: taint
4   pattern-sources:
5   - patterns:
6     - pattern: event
7     - pattern-inside: |
8       def $HANDLER(event, context):
9         ...
10  pattern-sinks:
11  - patterns:
12    - pattern-either:
13      - pattern: eval($CODE, ...)
14      - pattern: exec($CODE, ...)
15  message: >-
16    Detected the use of `exec/eval`. This can be dangerous if used to evaluate
17    dynamic content. If this content can be input from outside the program,
18    this may be a code injection vulnerability. Ensure evaluated content is
19    not definable by external sources.
20  languages:
21  - python
22  severity: WARNING
```

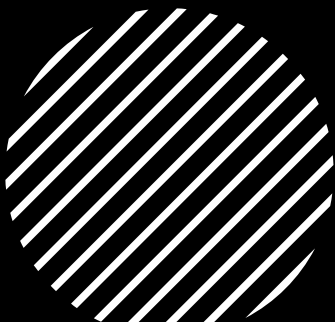


```
1 def handler(event, context):
2   # ok:tainted-code-exec
3   exec("x = 1; x = x + 2")
4
5   blah1 = "import requests; r = requests.get('https://example.com')"
6   # ok:tainted-code-exec
7   exec(blah1)
8
9   dynamic1 = "import requests; r = requests.get('{}')"
10  # ruleid:tainted-code-exec
11  exec(dynamic1.format(event['url']))
12
13  # ok:tainted-code-exec
14  eval("x = 1; x = x + 2")
15
16  blah2 = "import requests; r = requests.get('https://example.com')"
17  # ok:tainted-code-exec
18  eval(blah2)
19
20  dynamic2 = "import requests; r = requests.get('{}')"
21  # ruleid:tainted-code-exec
22  eval(dynamic2.format(event['url']))
```



LLM-based Code Analysis

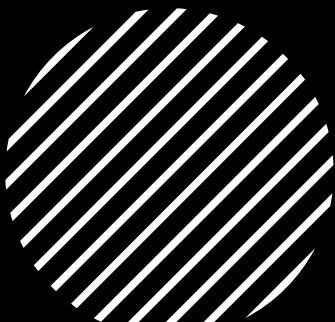
- SAST triage can be painful
 - Scale
 - Context
 - Confidence
- Leverage LLM capabilities
 - At scale
 - With context
 - Increased confidence





LLM-based Code Analysis

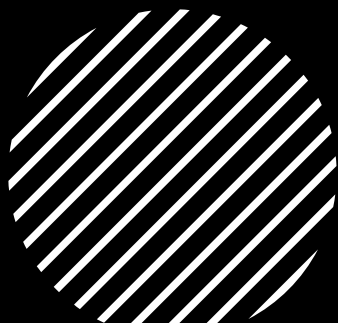
- SAST results can be exported via SARIF
- LLMs excel at (code) summarization
- Some LLMs have a "JSON constrained" output guarantee
- LLMs exhibit understanding the JSON spec





Building it up

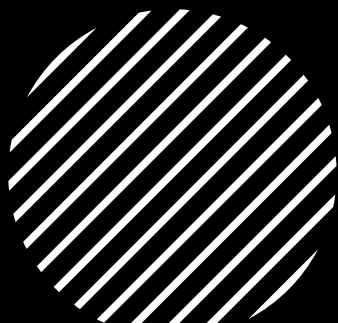
- SARIF offers interoperability between SAST tools
- SARIF is JSON
- Lots of language support to parse JSON





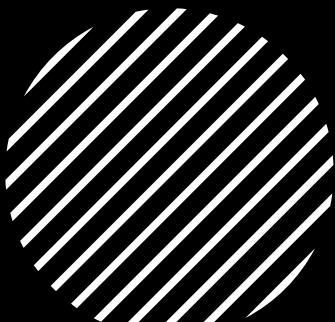
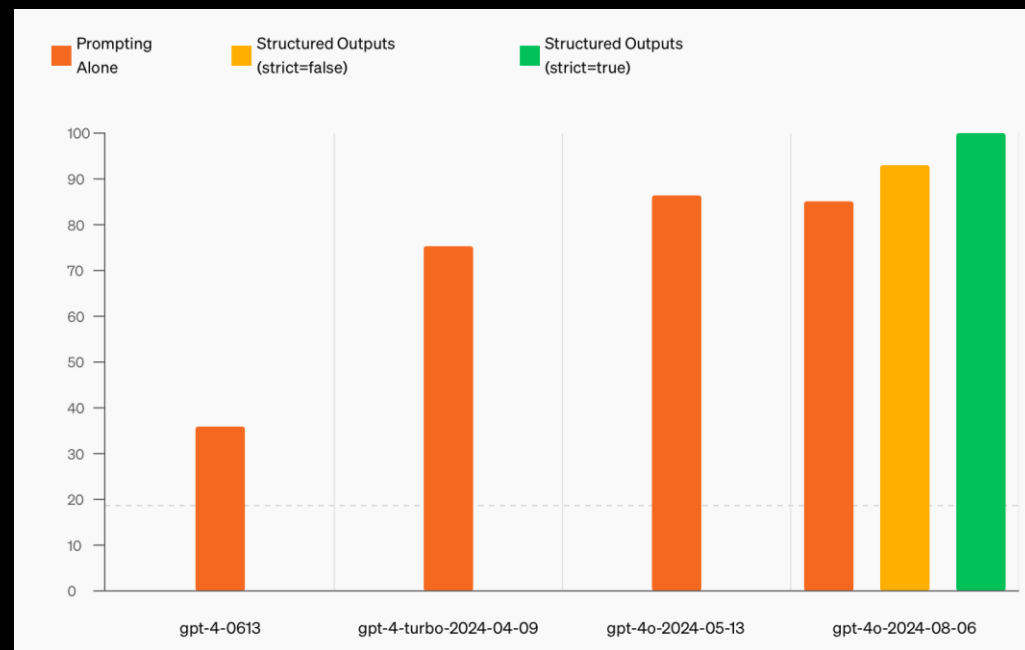
Building it up

- LLMs excel at summarization
- LLMs are also good at code generation
- LLMs need grounding and good prompts



Building it up

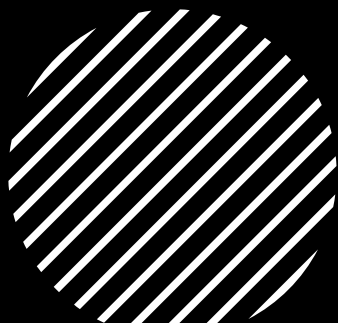
- LLMs love to talk
- LLMs can also be constrained to respond in certain manner, but with minimal guarantee of compliance
- Some LLMs have a “JSON constrained” output guarantee
 - Structured Output Mode





Building it up

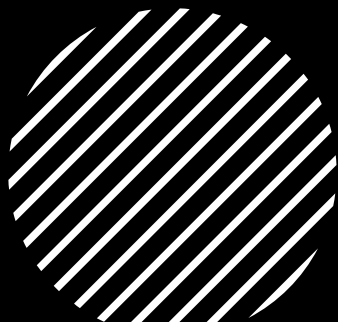
- LLMs exhibit understanding the JSON spec
- Redux: JSON is great for parsing
- Constraining output to JSON allows for structured summarization





Building it up

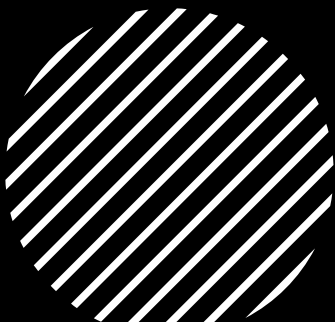
- Prompting is part art/science
- Persona + Scenario + Output constraint + Input to analyze
- More “context” increases accuracy of LLM output
 - Custom rules to output full class body
 - Fetch file content from URL



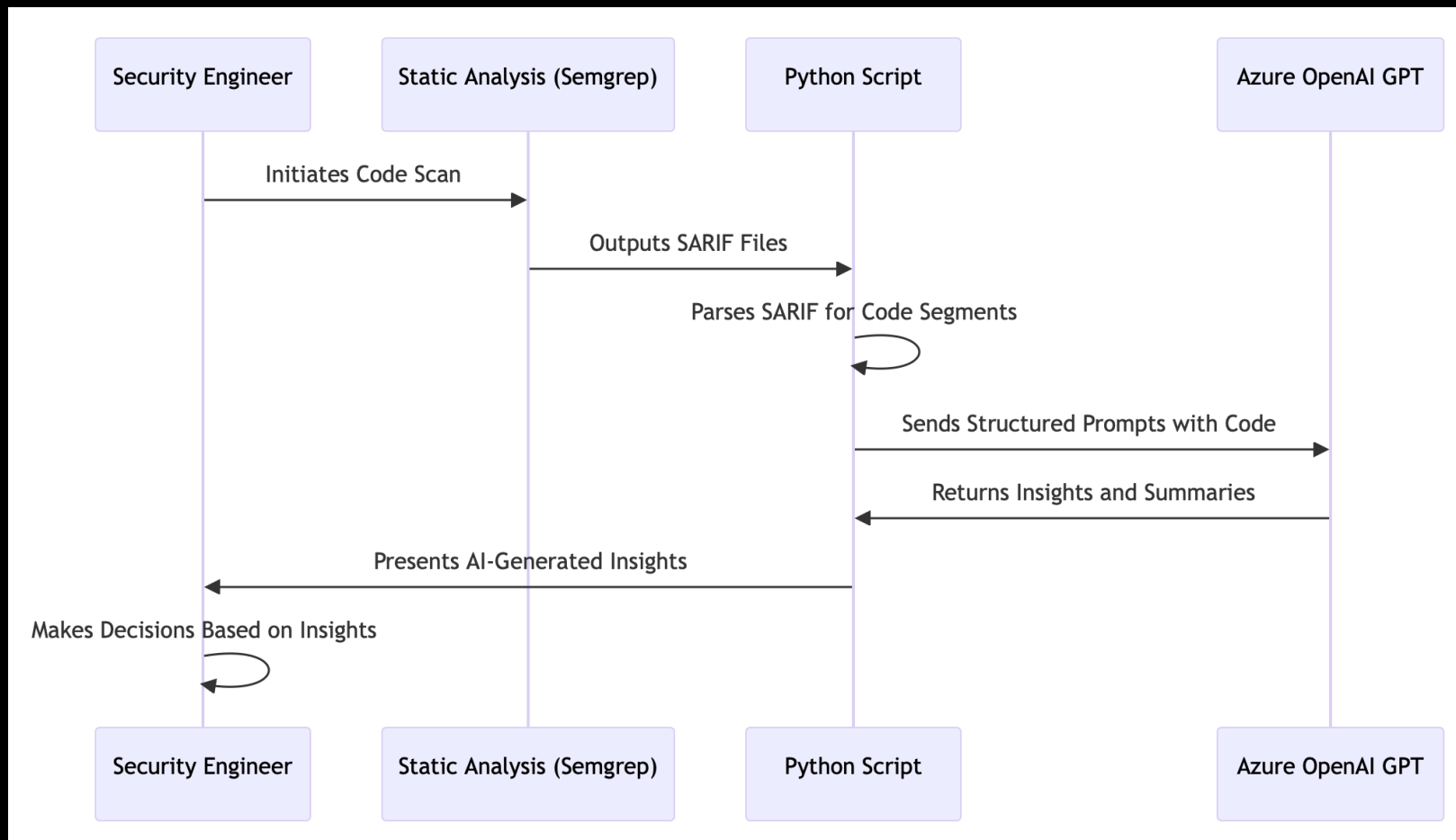


Putting it together

- Semgrep scan
 - Outputs SARIF
- SARIF (JSON) Parse
 - Extract code snippet
- Prompt GPT
 - System Prompt + JSON constraint on output + Code to analyze
- GPT Response
 - JSON output



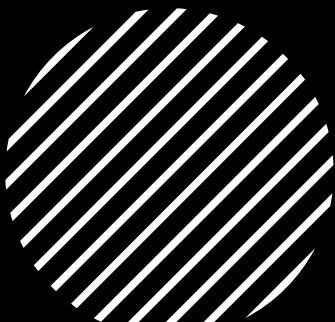
Putting it together



Putting it together

- Prompt
 - Basic

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "RegEx Usage Information",
  "description": "Schema for information about RegEx usage in a C# class, including security risk and details of each RegEx object",
  "type": "object",
  "properties": {
    "regexRisk": {
      "type": "string",
      "description": "The overall security risk of RegEx usage in this class",
      "enum": ["Low", "Medium", "High"]
    },
    "regexObjects": {
      "type": "array",
      "description": "Represents all objects in the class of C# type `Regex`",
      "items": {
        "$ref": "#/definitions/regexObject"
      }
    }
  }
},
```



Putting it together

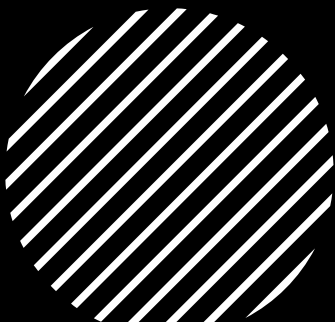
- Prompt
 - Better

```
"definitions": {
  "regexObject": {
    "type": "object",
    "properties": {
      "regexVariable": {
        "type": "string",
        "description": "The name of a C# variable of type `Regex`"
      },
      "regexIntendedUsage": {
        "type": "string",
        "description": "Summarize the usage of each variableNames entry, focusing on what the pattern is meant to represent based on variable name and usage scenario"
      },
      "regexActualUsage": {
        "type": "string",
        "description": "Summarize the usage of each variableNames entry, focusing on what the pattern actually represents as defined in the object constructor"
      },
      "regexTags": {
        "type": "string",
        "description": "The tags representing what the `regexVariable` is being used for. Can be multiple tags, comma-separated",
        "enum": [
          "input_validation",
          "parsing",
          "string_search",
          "string_split",
          "string_replace",
          "pattern_match",
          "url_parsing"
        ]
      },
      "isComplete": {
        "type": "boolean",
        "description": "Represents whether the regex pattern is a complete representation of the usage scenario, or can be security hardened or made more complete against attacker bypasses"
      }
    },
    "required": ["regexVariable", "regexIntendedUsage", "regexActualUsage", "regexTags", "isComplete"],
    "description": "Represents an instance of the an `Regex` object."
```

Putting it together

- Prompt
 - Even better (conditional sub-schemas)

```
"if": {
  "properties": { "isComplete": { "const": false } }
},
"then": {
  "properties": {
    "completenessRecommendation": {
      "type": "object",
      "properties": {
        "completenessExplanation": {
          "type": "string",
          "description": "A human-readable explanation of why the regex needs to be modified for completeness. If there is a security issue from its incomplete implementation, call out what security risks the current implementation poses (list out bug classes and attack types)"
        },
        "completeRegex": {
          "type": "string",
          "description": "The code for the modified regex constructor argument"
        }
      }
    },
    "required": ["completenessExplanation", "completeRegex"],
    "description": "Recommendations for making the regex pattern complete"
  }
},
"required": ["completenessRecommendation"]
```

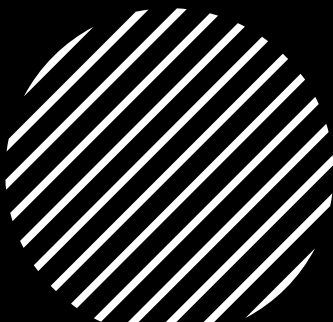




Putting it together

- Response

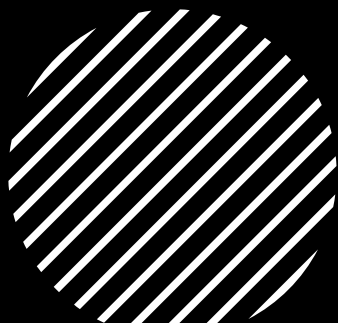
```
1 {
2   "regexRisk": "Medium",
3   "regexObjects": [
4     {
5       "regexVariable": "PathAndQueryRegex",
6       "regexIntendedUsage": "To separate the path and query parts of a URL",
7       "regexActualUsage": "Matches any characters that are not a question mark for the path, and optionally matches a query string starting with a question mark",
8       "regexTags": "url_parsing",
9       "isComplete": false,
10      "completenessRecommendation": {
11        "completenessExplanation": "The regex does not validate the URL structure rigorously. It assumes any string with or without a '?' character is a valid URL, which might not be the case. This could lead to security issues if used for security-sensitive operations without further validation, such as open redirects or SSRF vulnerabilities.",
12        "completeRegex": "@^(?<path>[^?]+)(?<query>\\?.*)?$"
13      }
14    },
15    {
16      "regexVariable": "invalidUrlPattern",
17      "regexIntendedUsage": "To identify parts of a URL that are potentially harmful or invalid",
18      "regexActualUsage": "Combines several patterns to match against URL parts that contain certain special characters, consecutive dots, ending with a dot, or leading/trailing spaces",
19      "regexTags": "input_validation,string_search",
20      "isComplete": false,
21      "completenessRecommendation": {
22        "completenessExplanation": "The dynamic construction of the regex based on the 'allowTokens' parameter could lead to regex injection if the parameter is controlled by user input. Additionally, the regex checks are somewhat superficial and do not fully ensure URL safety, potentially missing other forms of URL manipulation or encoding attacks.",
23        "completeRegex": "@[#%<:~\\|/{}?\\*\\+\\.\\$!@\\'\"<br>@["
24      }
25    }
26  ]
27 }
```

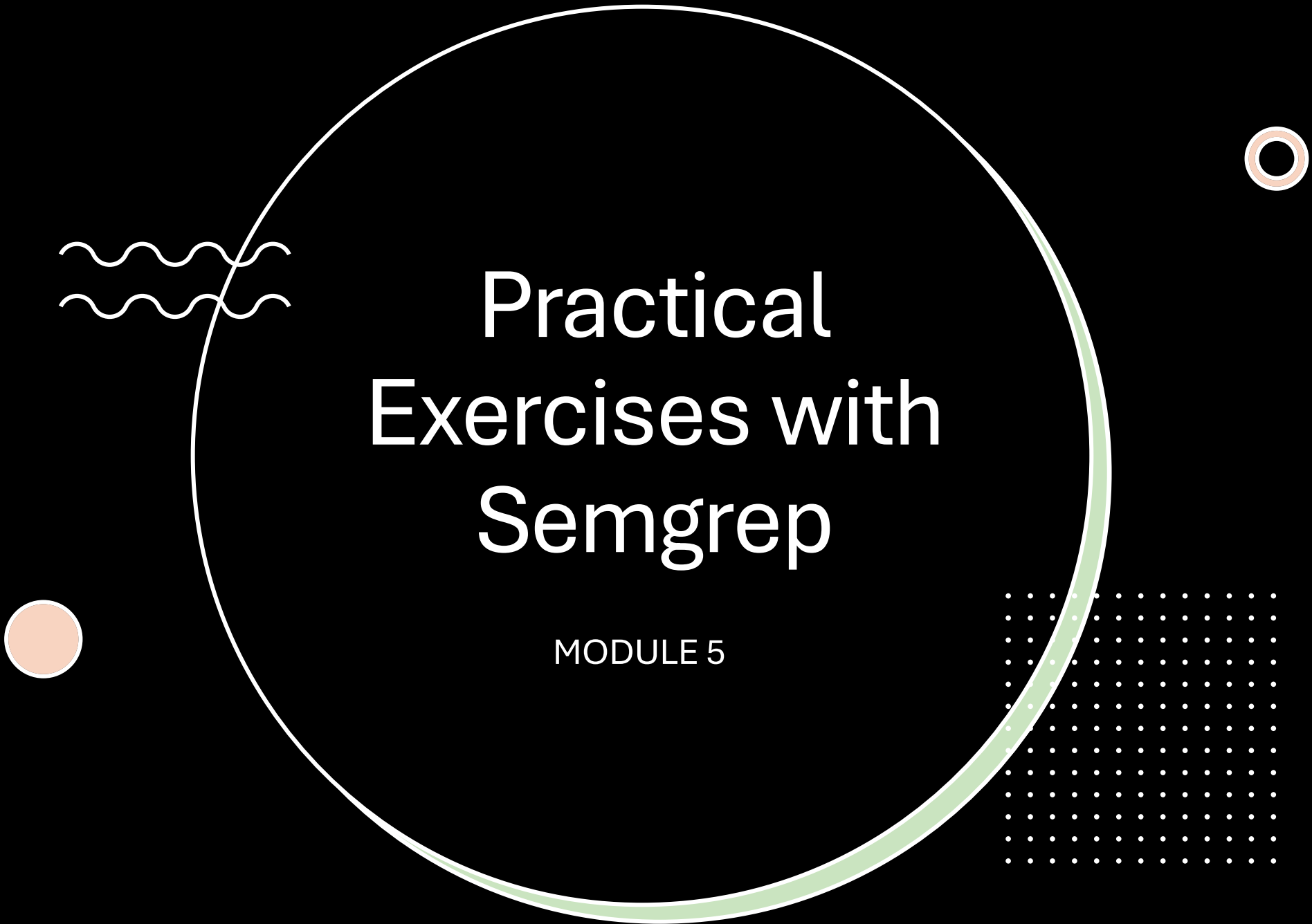




Initial Results

- Prioritized Triage
 - Riskiest <5%
- True Positives
 - >90%
- False Negatives
 - Harder to determine



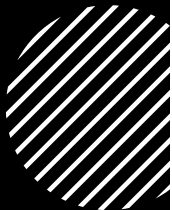


Practical Exercises with Semgrep

MODULE 5



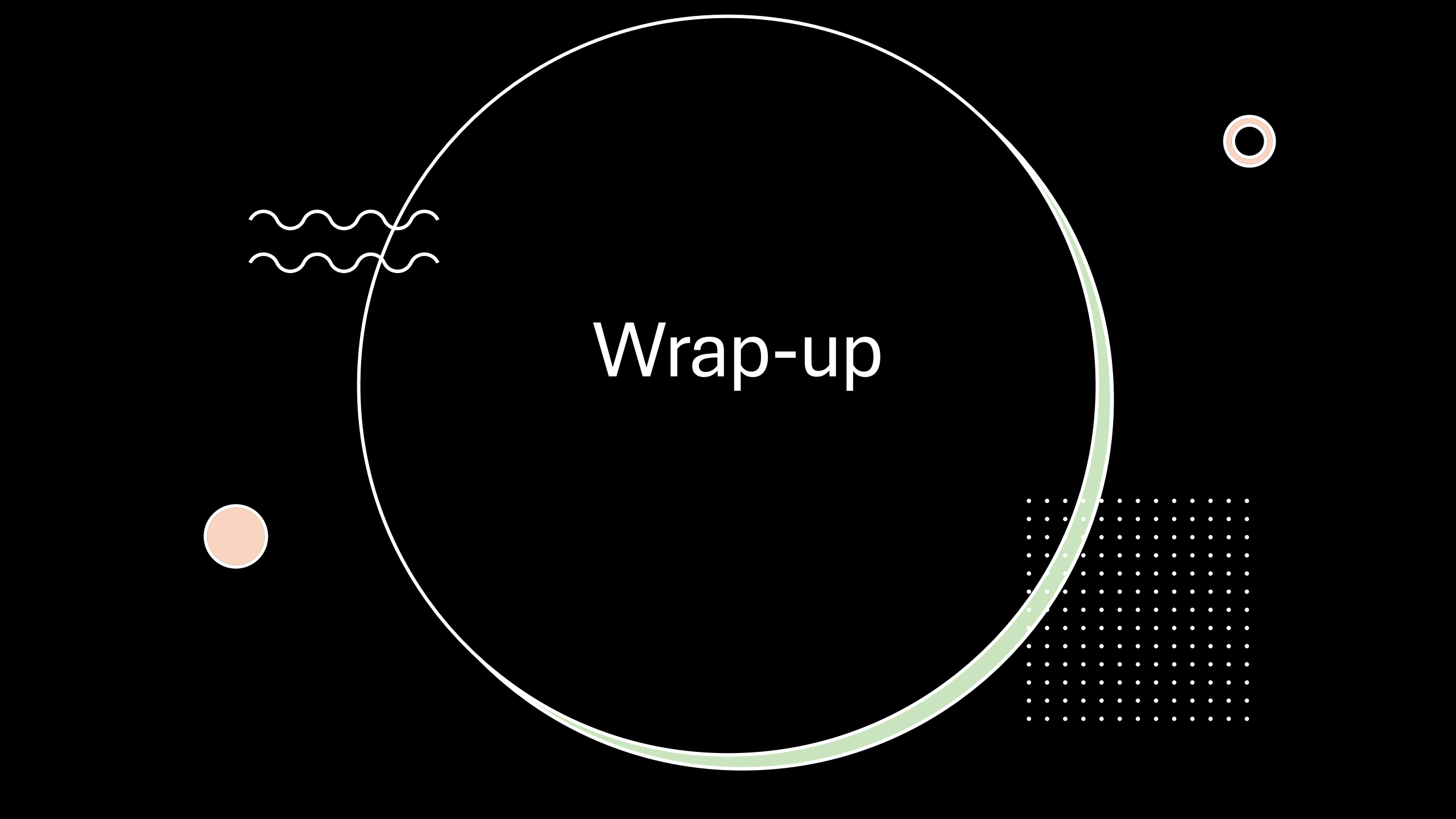
Practical Exercises with Semgrep



C#, TypeScript, Python



<https://aka.ms/semgrep-workshop>

The image features a central white circle with a thick green border. Inside this circle, the text "Wrap-up" is written in a white, sans-serif font. Surrounding the central circle are several decorative elements: two white wavy lines to the left, a small orange circle with a white outline to the bottom-left, a small orange circle with a white outline to the top-right, and a grid of small white dots to the bottom-right.

Wrap-up



Wrap-up



Recap of key takeaways
from the workshop

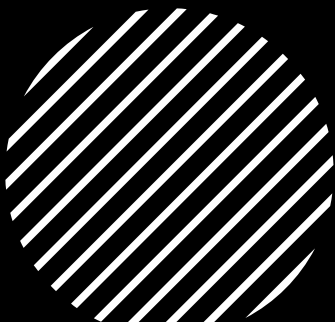


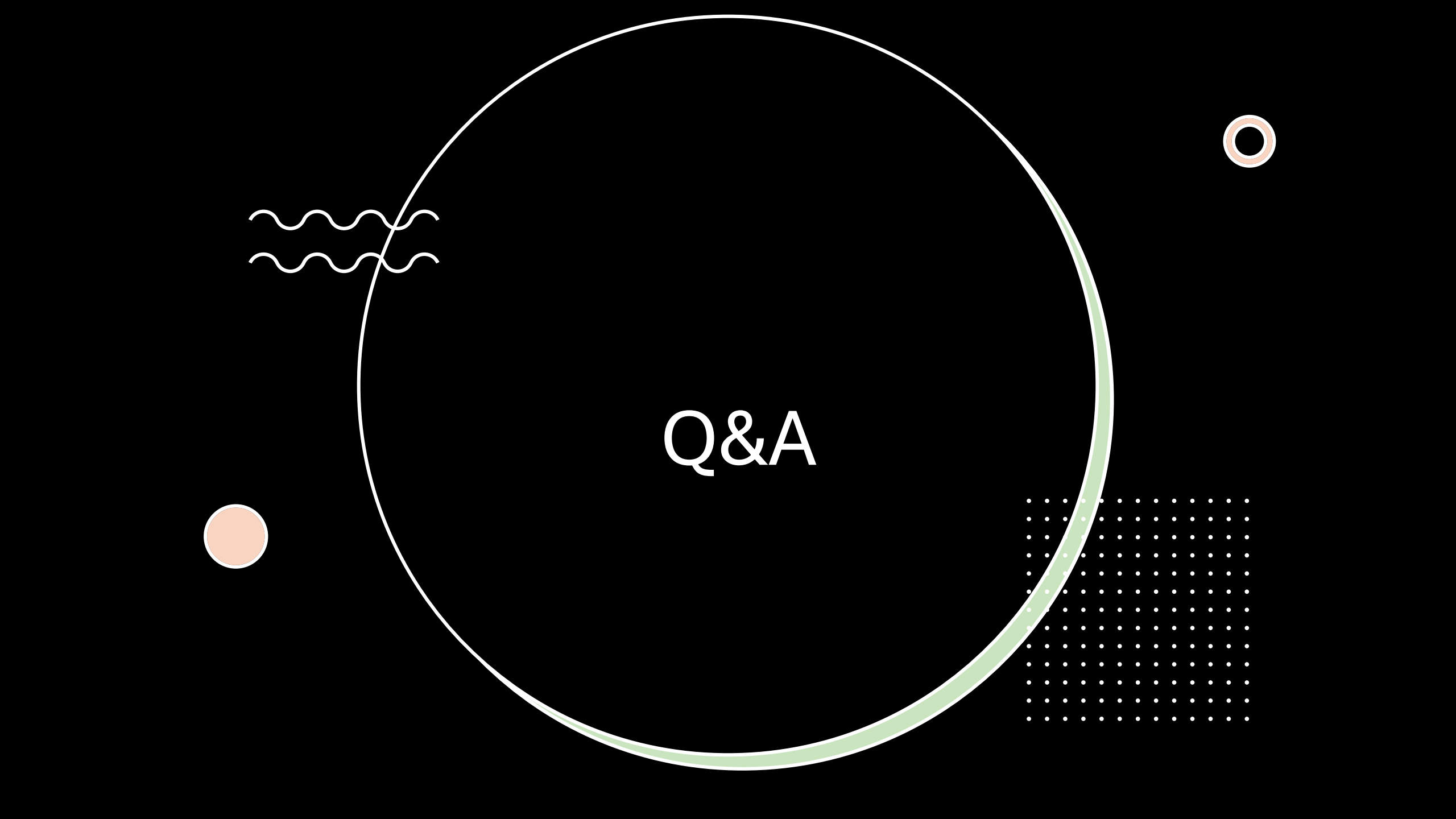
Learning resources and
contributing to community



References

- Academy Courses
 - [Semgrep 101](#)
 - [Secure Coding](#)
 - [Semgrep Custom Rules Level 1](#)
- Semgrep Documentation
 - [Quickstart](#)
 - [Local scans with Semgrep](#)
 - [Writing rules](#)
 - [Custom rule examples](#)



The image features a large, thin white circle centered on a black background. Inside this circle, the text "Q&A" is written in a white, sans-serif font. Surrounding the central circle are several decorative elements: a thick green arc on the right side of the circle, a small orange circle with a white outline on the left, a small orange ring with a white outline in the top right, and a grid of small white dots in the bottom right. On the left side, outside the main circle, are two horizontal wavy lines.

Q&A